

PharmaCare Technical Manual

Developer Reference

Version 1.0 — 2026

Section 1 — System Overview

1.1 Technology Stack

Layer	Technology
Frontend framework	Next.js 16 (App Router), TypeScript
Styling	Tailwind CSS v4
Charts	Recharts
Icons	Lucide React
Backend / database	Supabase (PostgreSQL + Auth + Row Level Security + Storage)
Test framework	Jest (not Vitest)
Deployment	Vercel

1.2 Architecture Overview

- **Role-prefixed route structure:** `/superadmin/*`, `/admin/*`, `/pharmacist/*`. Each role tree has its own `layout.tsx` that verifies the session and role, resolves the caller's permission set, and renders a role-specific shell and sidebar. There is no shared, role-branching layout attempting to handle all three roles.
- **Server Actions for all mutations.** Data-writing operations are implemented as `'use server'` functions in `app/actions/*.ts`, called directly from Server Components or client components — not via hand-rolled REST API routes.
- **Row Level Security (RLS) policies** enforce access control at the database layer, independently of the application layer, so a compromised or bypassed frontend still cannot read or write data a role is not permitted to touch.
- **Cookie-based authentication** via `@supabase/ssr` — session state lives in cookies managed by the Supabase SSR helpers, not in `localStorage`.

1.3 Project Structure — Key Files

File	Purpose
<code>lib/permissions.ts</code>	Single source of truth for the permission model: the <code>PERMISSIONS</code> map, per-role base permission sets, <code>resolvePermissions()</code> , and <code>hasPermission()</code>
<code>lib/dashboard-context.tsx</code>	Client-side context (<code>DashboardUserContext</code>) exposing the current user's id, role, and fully-resolved permission array to any client component via <code>useDashboardUser()</code>
<code>lib/print-utils.ts</code>	Single source of truth for all business-document printing — <code>printDocument()</code> builds a complete standalone HTML document and opens it in a popup window for printing, avoiding the overflow/clipping problems of in-page <code>@media print</code>
<code>lib/pos-shortcuts.ts</code>	Single source of truth for POS keyboard shortcuts — 26 entries across 5 categories (<code>sale</code> , <code>navigation</code> , <code>cart</code> , <code>modal</code> , <code>wizard</code>), plus DOM helpers for quantity-field focus management
<code>lib/audit.ts</code>	Audit logging helper used across write-path server actions
<code>app/actions/*.ts</code>	All server actions — 17 files: <code>audit.ts</code> , <code>auth.ts</code> , <code>borrowing.ts</code> , <code>exchange-rates.ts</code> , <code>expenses.ts</code> , <code>item-report.ts</code> , <code>ledger.ts</code> , <code>medicines.ts</code> , <code>procurement.ts</code> , <code>reports.ts</code> , <code>returns.ts</code> , <code>sales.ts</code> , <code>settings.ts</code> , <code>shifts.ts</code> , <code>stock.ts</code> , <code>suppliers.ts</code> , <code>users.ts</code>
<code>proxy.ts</code>	Next.js middleware — route guards per role prefix, plus <code>force_password_change</code> enforcement on every request including Server Action POSTs
<code>supabase/migrations/</code>	36 sequential SQL migrations (001 through 036) — see Section 7

Section 2 — Database Schema

2.1 Core Tables

34 tables exist in the `public` schema of the live database:

accounts, audit_logs, borrowing_pharmacies, borrowing_transactions,
 controlled_drug_register, customer_payments, customers,
 daily_reconciliations, doctors, exchange_items, exchange_rates,
 expenses, generic_names, goods_receipts, grn_items, journal_entries,
 journal_lines, medicine_categories, medicine_subcategories, medicines,
 prescriptions, profiles, purchase_order_items, purchase_orders,
 return_items, returns, sale_items, sales, settings, shifts,
 stock_batches, supplier_payments, suppliers, user_permissions

Selected tables in detail:

- **profiles** — extends `auth.users`. Key columns: `role` (`superadmin` / `admin` / `pharmacist` / `pending`), `is_active`, `force_password_change`, `special_discount_max_tier`, `can_perform_daily_close`. One row per authenticated user, `id` shared with `auth.users.id`.
- **medicines** — master catalog. Key columns: `name`, `generic_name`, `manufacturer`, `schedule` (CHECK `'OTC' | 'prescription' | 'controlled'`), `mrp`, `reorder_level`, `category_id` / `subcategory_id` FKs, `generic_name_id` FK. No boolean `requires_prescription` / `is_controlled` columns — `schedule` alone gates behavior.
- **stock_batches** — batch-level inventory. Key columns: `medicine_id`, `batch_no`, `expiry_date`, `quantity` (CHECK `>= 0`), `purchase_price`, `sale_price`, `mrp` (batch-level override; both nullable — POS falls back to the medicine's own `mrp` when null), `supplier_id`, `grn_id`. Unique on (`medicine_id`, `batch_no`).
- **sales** / **sale_items** — `sales.payment_type` CHECK allows `cash` / `credit` / `bank_transfer` / `cheque` (widened in migration 035). `sale_items.unit_price` is validated against the batch/medicine `mrp` inside `complete_sale()`.
- **purchase_orders** — `status` CHECK: `draft`, `pending_approval`, `confirmed`, `partially_received`, `received`, `cancelled`, `closed_short`. `po_number` defaults from `next_po_number()` (date-based, not transaction-safe under concurrency — acceptable for single-branch volume).
- **goods_receipts** / **grn_items** — GRN header/lines. `grn_items` has no `sale_price` / `mrp` columns — pricing for a received batch must be set separately (via the batch edit form) after receipt.
- **journal_entries** / **journal_lines** — the double-entry ledger core. See Section 2.3 for immutability rules.
- **returns** / **return_items** / **exchange_items** — the returns/exchange workflow, including policy-flag tracking (`window_expired`, `opened_pack`, `exceeds_limit`) and optional linked exchange sale.
- **shifts** — `status` CHECK `'open' | 'closed'`, plus Phase 12 columns (`policy_type`, `scheduled_start/end`, `reconciled`, `transferred_at/by`) that exist in schema but are feature-flagged off by default.

- `user_permissions` — override table only. One row per `(user_id, permission)` with `type` `'grant'` or `'restrict'`. Base permission sets live in code (`lib/permissions.ts`), not in this table.

2.2 Chart of Accounts

26 active accounts:

1000 Cash	asset / debit	
1001 Bank Account	asset / debit	
1100 Accounts Receivable	asset / debit	
1110 Borrowing Receivable	asset / debit	
1200 Inventory	asset / debit	
1300 Prepaid Expenses	asset / debit	
2000 Accounts Payable	liability / credit	
2010 Borrowing Payable	liability / credit	
2100 Customer Deposits	liability / credit	
3000 Owner Equity	equity / credit	
3100 Retained Earnings	equity / credit	
4000 Sales Revenue	revenue / credit	
4010 Other Revenue	revenue / credit	
4800 Cash Overage Income	revenue / credit	
4900 Sales Discount	revenue / debit	← contra-revenue, see 5.3
5000 Cost of Goods Sold	cogs / debit	
6000 Operating Expenses	expense / debit	
6001 Electricity	expense / debit	
6002 Rent	expense / debit	
6003 Salaries	expense / debit	
6004 Fuel & Transport	expense / debit	
6005 Maintenance & Repairs	expense / debit	
6006 Internet & Communication	expense / debit	
6007 Printing & Stationery	expense / debit	
6008 Other Expenses	expense / debit	
6800 Cash Shortage Expense	expense / debit	

Note account 4900: it shares `account_type = 'revenue'` with 4000/4010 but has `normal_balance = 'debit'` — it is a contra-revenue account. This matters directly to the known bug in Section 5.3.

2.3 Key Constraints

- **journal_lines immutability** — trigger `journal_lines_immutable` blocks all UPDATE and DELETE on the table, unconditionally. Rows are append-only for the life of the database.
 - **journal_entries_protect_posted trigger** (function `prevent_posted_entry_mutation()`) — once `status IN ('posted', 'reversed')`, blocks any change to `entry_date`, `description`, `reference_type`, `reference_id`, `currency`, or `exchange_rate`. The only permitted post-posting mutation is `status → 'reversed'` plus setting `reversed_by`, done via `mark_entry_reversed()`.
 - **audit_logs immutability trigger** — added in migration 036 (BUG-3), preventing UPDATE/DELETE on audit log rows.
 - **purchase_orders status transition trigger** — added in migration 036 (BUG-5) to enforce valid state transitions at the database layer (previously enforced only loosely at the application layer).
 - **complete_sale() shift requirement** — the function raises an exception unless `EXISTS (SELECT 1 FROM shifts WHERE cashier_id = p_cashier_id AND status = 'open')`. Note this check is by cashier existence of *any* open shift, not tied to the specific calendar date of the sale.
 - **idx_journal_entries_single_opening_balance** — a partial unique index on `journal_entries(reference_type) WHERE reference_type = 'opening_balance'`, guaranteeing only one opening-balance entry can ever exist.
 - **medicines_select RLS landmine** — the SELECT policy filters `is_deleted = false`, and PostgreSQL applies that filter to the *post-update* row state even for UPDATE statements. Any authenticated-client UPDATE that sets `is_deleted = true` on a medicine will fail with an RLS violation, because the resulting row fails its own SELECT policy. Soft-deleting a medicine must go through a service-role call or a `SECURITY DEFINER` RPC.
-

Section 3 — Authentication & Authorization

3.1 Auth Flow

- Sessions are managed via `@supabase/ssr` cookie-based auth — not `localStorage`, not custom JWT handling.
- `proxy.ts` (the Next.js middleware) checks the session on every request, enforces `force_password_change` (including on Server Action POSTs, not just page navigations), and applies role-based route guards keyed on path prefix (`/superadmin`, `/admin`, `/pharmacist`).
- API keys use the publishable/secret key system (`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, not the legacy `NEXT_PUBLIC_SUPABASE_ANON_KEY`, though the legacy key may still exist unused in

`.env.local`).

- JWT expiry is 3600 seconds (1 hour); a separate client-side inactivity timer (30 minutes, configurable via settings) handles idle-session logout independently of token expiry.

3.2 Permission System

`lib/permissions.ts` defines **18 permissions**:

```
users_manage, user_manage_pharmacists, settings, audit_trail,
reports_full, reports_basic, expenses, suppliers, purchase_orders,
inventory_view, inventory_manage, pos, sales_history_own,
sales_history_all, prescriptions, controlled_drugs, customers, shifts
```

SUPERADMIN_PERMISSIONS — all 18, hardcoded, no per-user override applies (SuperAdmin always gets the full set).

ADMIN_BASE_PERMISSIONS (current live set — note this has grown since the original RBAC V2 spec draft): `suppliers` , `purchase_orders` , `inventory_view` , `inventory_manage` , `customers` , `shifts` , `expenses` , `reports_full` . (`expenses` and `reports_full` were added to this base set in a later phase — the original spec had them as SuperAdmin-grantable extras only.)

PHARMACIST_BASE_PERMISSIONS : `pos` , `prescriptions` , `controlled_drugs` , `customers` , `shifts` , `inventory_view` , `sales_history_own` , `reports_basic` .

resolvePermissions(role, overrides) — for `admin` / `pharmacist` , starts from the role's base set, removes any `restrict` -type override, then adds any `grant` -type override. For `superadmin` , returns the full set unconditionally regardless of any override rows present.

hasPermission(resolvedPermissions, permission) — a plain array-includes check against an already-resolved permission list; it does not re-fetch or re-resolve anything.

Overrides live in the `user_permissions` table as `(user_id, permission, type)` rows — the base sets themselves are never stored in the database, only deviations from them.

3.3 RLS Policies

- Standard per-table pattern: `SELECT` policies gated on `get_user_role() IN (...)` plus `is_deleted = false` ; `INSERT` / `UPDATE` similarly role-gated; no `DELETE` policy on any business table (soft-delete only).
- `audit_logs` : `INSERT` for all authenticated roles, `SELECT` for admin/superadmin, no `UPDATE/DELETE` policy at all (and, since migration 036, a trigger reinforcing this even against a

service-role bypass attempt).

- `controlled_drug_register` : INSERT/SELECT for pharmacist/admin/superadmin; no UPDATE/DELETE policy — append-only by design.
- `post_journal_entry()` is `SECURITY DEFINER` and bypasses RLS by design — it must only ever be called from server actions or other RPCs, never directly from client-side code.
- **Known RLS gap:** `purchase_order_items` DELETE policy is role-only; the "draft status only" guard is enforced at the application layer (`removePOItem()`), not at the RLS layer — a direct API call could bypass it.

Section 4 — Core RPCs

All 10 signatures below were read directly from the live database (`pg_get_functiondef`), not assumed.

`complete_sale(p_cashier_id uuid, p_customer_id uuid, p_payment_type text, p_items jsonb, p_discount_amt numeric, p_bag_charge numeric, p_amount_paid numeric, p_notes text) RETURNS jsonb`

Validates an open shift exists for the cashier, validates stock sufficiency and non-expired batches, enforces `unit_price <= MRP` per item, inserts `sales` / `sale_items` , decrements `stock_batches.quantity` , computes COGS from batch `purchase_price` , and posts a journal entry: debit the payment account (1000 cash / 1001 bank-or-cheque / 1100 credit) for the total, credit 4000 for the gross subtotal, debit 4900 for any discount, credit 4010 for any bag/service charge, and debit 5000 / credit 1200 for COGS. `entry_date` is hardcoded to `CURRENT_DATE` — there is no date parameter.

`complete_grn(p_po_id uuid, p_received_by uuid, p_notes text, p_items jsonb, p_is_partial boolean DEFAULT false) RETURNS uuid` Requires the PO to be in `confirmed` or `partially_received` status.

Inserts the GRN header and `grn_items` , upserts `stock_batches` (`ON CONFLICT (medicine_id, batch_no) DO UPDATE SET quantity = quantity + EXCLUDED.quantity`), updates PO status to `received` or `partially_received` based on `p_is_partial` , and — only if the total value is greater than zero — posts DEBIT 1200 / CREDIT 2000 (with `party_type='supplier'` / `party_id`). `entry_date` is hardcoded to `CURRENT_DATE` .

`process_return(p_original_sale_id uuid DEFAULT NULL, p_return_items jsonb DEFAULT NULL, p_exchange_items jsonb DEFAULT NULL, p_reason text DEFAULT NULL, p_pack_opened boolean DEFAULT false, p_requested_by uuid DEFAULT NULL, p_return_id uuid DEFAULT NULL) RETURNS jsonb` A dual-mode function. Mode A (`p_return_id IS NULL`): validates the return, blocks controlled-substance returns outright, checks for double-returns, evaluates policy (return window, opened-pack rule, auto-approval limit) and either completes immediately (`auto_approved`) or stops at `pending_approval` . Mode B (`p_return_id` provided): loads the pending return, marks it approved, and falls through to the same reversal logic as Mode A's auto-approved path. The reversal restores stock to the exact

original batch(es), reverses COGS, reverses the proportional slice of any original discount, and posts the balanced reversal journal entry. `entry_date` is hardcoded to `CURRENT_DATE`.

`post_journal_entry(p_entry_date date, p_description text, p_reference_type text, p_reference_id uuid, p_currency text, p_exchange_rate numeric, p_lines jsonb, p_created_by uuid) RETURNS uuid`

The foundational primitive every other accounting RPC calls internally. Generates a sequential `entry_no`, inserts the `journal_entries` header as `status='posted'`, inserts every line in `p_lines` into `journal_lines`, and — critically — raises an exception if `ABS(debit_sum - credit_sum) >= 0.0001` before returning. This is the only RPC of the ten that accepts `p_entry_date` as a genuine caller-supplied parameter rather than hardcoding `CURRENT_DATE`.

`record_customer_payment(p_customer_id uuid, p_amount numeric, p_payment_method text, p_reference_no text, p_notes text, p_recorded_by uuid) RETURNS uuid` Atomic RPC (fully resolving what was previously a known ordering gap). Validates the customer exists, rejects the payment if `p_amount` exceeds the customer's current `credit_balance`, inserts `customer_payments`, posts DEBIT 1000/1001 (by payment method) / CREDIT 1100 (with `party_type='customer'`), links the journal entry back to the payment row, then decrements `credit_balance`. `entry_date` is hardcoded to `CURRENT_DATE`.

`get_financial_summary(p_date_from date, p_date_to date) RETURNS TABLE(account_type text, total_amount numeric)` Aggregates `journal_lines` joined to `journal_entries` / `accounts`, filtered to `status IN ('posted','reversed')` and the date range, grouped by `account_type ∈ {'revenue','cogs','expense'}`, applying each account's own `normal_balance` sign convention. See Section 5.3 for a known defect in this aggregation.

`get_balance_sheet(p_as_of_date date) RETURNS TABLE(section text, account_code text, account_name text, account_type text, balance numeric, display_order int)` All-time asset/liability/equity balances as of the given date (non-zero only), plus a synthetic `NET` row (`account_code='NET'`, `display_order=999`) computed correctly as `4xxx credit-normal revenue - 4900 debit-normal contra-revenue - COGS - Expenses` for the fiscal year to date. This function's own `NET` calculation does **not** have the 4900 sign bug that `get_financial_summary()` has.

`get_trial_balance(p_from date, p_to date) RETURNS TABLE(account_code, account_name, account_type, normal_balance, total_debits, total_credits, net_balance, has_activity)` Every account, including zero-activity ones (shown as zeros via `LEFT JOIN`, with `has_activity=false`), with debit/credit totals for the date range.

`get_cash_book(p_date date) RETURNS TABLE(entry_time, entry_id, entry_no, description, in_amount, out_amount, opening_balance, running_balance)` Takes a single date, not a range. Returns every journal line touching account 1000 for that day, in order, with a running balance that accumulates correctly across the day (the `opening_balance` value is repeated identically on every row as a reference figure — this is a display convention, not a computation error).

`get_party_ledger(p_party_type text, p_party_id uuid, p_date_from date DEFAULT NULL, p_date_to date DEFAULT NULL) RETURNS TABLE(entry_id, entry_date, entry_no, description, account_code, account_name, debit_amount, credit_amount, running_balance)` Filters `journal_lines` by `party_type` / `party_id` (populated on the AR/AP lines of the relevant RPCs above), within an optional date range, with a running balance.

Error conditions common to all of the above: every one of these RPCs is `SECURITY DEFINER`. Attempting to call `post_journal_entry()` (or any RPC that wraps it) with unbalanced lines raises `'Journal entry does not balance. Debits: %, Credits: %'`. Attempting to reference a non-existent or inactive account code raises `'Account not found or inactive: %'`.

Section 5 — Accounting System

5.1 Double-Entry Rules

- Every transaction produces a balanced journal entry — enforced inside `post_journal_entry()` itself, not by a separate validation pass.
- `journal_lines` are immutable; there is no code path, including admin tooling, that updates or deletes a posted line under normal operation.
- Reversal pattern: a *new*, equal-and-opposite journal entry is posted (never an edit to the original), then `mark_entry_reversed(p_original_id, p_reversal_id)` flips the original's `status` to `'reversed'` and links the two entries via `reversal_of`.
- Payment routing is consistent across every RPC that handles money: `cash → 1000`, `bank_transfer` / `cheque → 1001`, `credit → 1100` (AR) or `2000` (AP) depending on direction. No function hardcodes `1000` for a non-cash payment type.

5.2 Account Code Reference

See Section 2.2 for the full chart of accounts with codes, names, and normal balances.

5.3 Known Issues

`get_financial_summary()` — **4900 contra-revenue bug**. The function groups by `account_type` and applies each account's own `normal_balance` sign in isolation, then sums within the group. Because 4900 (Sales Discount) shares `account_type='revenue'` with 4000/4010 but has an opposite

`normal_balance` (`debit` vs `credit`), its balance is *added* to the revenue bucket instead of being subtracted from it. The practical effect: reported "revenue" from this function is inflated by roughly twice the total discount amount, relative to a correctly-netted figure (`4000 net - 4900 net` , ignoring 4010). This was independently verified against the demo dataset: raw function output showed revenue of PKR 1,403,196.06 against a manually-reconciled correct figure of PKR 1,373,403.94 — a difference of exactly PKR 29,792.12, i.e. 2× the 4900 net balance of PKR 14,896.06, the precise signature of this sign-handling defect. **Workaround:** trust `get_balance_sheet()` 's `NET` row instead, which computes the same figure correctly by explicitly subtracting the 4900 balance rather than relying on the generic per-account-type grouping. This is a distinct, separate issue from the Phase 16C reversal-timing double-count bug described below — it manifests even with zero returns present in the data, purely from the contra-account sign handling.

Reversal-timing double-count (Phase 16C, fixed in migration 036 — `BUG-1`). Report functions previously undercounted / double-counted when a reversal landed in the same query window as its original entry. This was one of the seven bugs fixed by migration 036 and is not currently an open issue.

Section 6 — Test Suite

6.1 Test Infrastructure

- **Jest**, not Vitest — this has been a deliberate, repeated decision throughout the project.
- Running more than one journal-writing test file together (`accounting.test.ts` + `inventory.test.ts`) **requires** `--runInBand` — Jest's default multi-worker mode runs test files as separate OS processes, and they genuinely race against each other on `post_journal_entry()` 's `entry_no` generation against the shared database. Single-file runs do not need this flag.
- `SUPABASE_DB_URL` (a direct Postgres connection string) is required for test cleanup, specifically because deleting test-created `journal_lines` / `journal_entries` rows requires temporarily disabling the `journal_lines_immutable` and `journal_entries_protect_posted` triggers inside a transaction, doing the exact-UUID-scoped delete, and re-enabling both triggers before commit — this cannot be done through the ordinary Supabase client.
- Test data uses a tracked-UUID cleanup pattern (`tests/helpers/test-client.ts`), not a filename/pattern-based cleanup, to avoid any risk of touching real data.

6.2 Test Files

File	Tests	Coverage
<code>route-access.test.ts</code>	148	Route-level RBAC guards (requires <code>npm run dev</code> on <code>localhost:3000</code> — the only suite with that dependency)
<code>rls-policies.test.ts</code>	54	Database RLS policy enforcement
<code>functional-flows.test.ts</code>	29	End-to-end functional flows
<code>accounting.test.ts</code>	62	Journal entries for <code>complete_sale</code> , <code>complete_grn</code> , <code>process_return</code> , <code>post_journal_entry</code> , <code>recordExpense</code> , <code>recordSupplierPayment</code> , <code>recordCustomerPayment</code> , borrowing
<code>inventory.test.ts</code>	30	Stock batch creation, FEFO behavior (app-layer only, not DB-enforced), <code>adjustStock()</code> / <code>writeOffBatch()</code> semantics
<code>business-rules.test.ts</code>	27	POS rules, PO status transitions, return policy, soft-delete, audit integrity, journal immutability, opening balances
<code>reports.test.ts</code>	12	Report function correctness
<code>smoke.test.ts</code>	7	End-to-end critical-path flows (one per major workflow)

Total: 369 tests across 8 files. (This differs from the original Phase 16 spec's ~383-test estimate — the spec's numbers were pre-implementation minimums, not a target to hit exactly; the final counts above are what was actually implemented and verified passing.)

6.3 Running Tests

```
npx jest tests/rls-policies.test.ts tests/functional-flows.test.ts \  
  tests/accounting.test.ts tests/inventory.test.ts \  
  tests/business-rules.test.ts tests/reports.test.ts \  
  tests/smoke.test.ts --runInBand
```

Expected: **221/221 passing** (everything except `route-access.test.ts` , which needs a running dev server and is run separately: `npx jest tests/route-access.test.ts` → 148/148).

Section 7 — Migration History

All 36 migrations, in sequence:

#	File	Summary
001	<code>initial_schema.sql</code>	Base schema
002	<code>fix_stock_summary_security.sql</code>	Security fix for the stock summary function
003	<code>smoke_test.sql</code>	Smoke-test seed data
004	<code>fix_rols_update_and_pending_access.sql</code>	RLS UPDATE policy + pending-role access fix
005	<code>fix_soft_delete_select_policy.sql</code>	Corrected soft-delete SELECT policy filtering
006	<code>rbac_v2.sql</code>	3-tier RBAC migration: role values, <code>user_permissions</code> table
007	<code>user_management_fields.sql</code>	Additional user management columns
008	<code>medicine_stock.sql</code>	Medicine categories/subcategories seed + stock schema
009	<code>supplier_procurement.sql</code>	Supplier/PO schema
010	<code>po_item_delete_policy.sql</code>	PO item delete policy correction
011	<code>pos.sql</code>	POS/sales schema
012	<code>ledger.sql</code>	Ledger DB foundation — accounts, journal tables, core RPCs
013	<code>update_rpcs_accounting.sql</code>	Wired <code>complete_sale</code> / <code>complete_grn</code> to post journal entries
014	<code>ledger_read_functions.sql</code>	Ledger read/reporting functions
015	<code>expenses.sql</code>	Expenses module
016	<code>report_functions.sql</code>	Initial report DB functions
017	<code>extended_report_functions.sql</code>	Additional report DB functions
018	<code>shifts.sql</code>	Shift management (column renames, RPC updates)
019	<code>returns_exchanges.sql</code>	Returns/exchanges schema
020	<code>process_return.sql</code>	<code>process_return()</code> RPC
021	<code>session_timeout_setting.sql</code>	Session timeout setting
022	<code>borrowing_pos.sql</code>	Inter-pharmacy borrowing integrated into POS
023	<code>generic_names.sql</code>	Generic names table

#	File	Summary
024	<code>partial_grn.sql</code>	Partial GRN support
025	<code>po_force_close.sql</code>	PO force-close (<code>closed_short</code> status)
026	<code>expense_void.sql</code>	Expense void workflow
027	<code>journal_reference_type_expense_void.sql</code>	Added <code>expense_void</code> to the <code>reference_type</code> CHECK
028	<code>item_detail_report_functions.sql</code>	Item-level detail report functions
029	<code>special_discount.sql</code>	Special discount permission system
030	<code>generic_alternatives_function.sql</code>	<code>get_generic_alternatives()</code> function
031	<code>phase12_shift_management.sql</code>	Extended shift-management schema (feature-flagged off by default)
032	<code>accounting_fixes.sql</code>	Restored <code>complete_grn</code> journal posting, fixed payment routing, separated discount into 4900
033	<code>financial_statements.sql</code>	<code>get_trial_balance</code> , <code>get_balance_sheet</code> , atomic <code>record_customer_payment</code> RPC
034	<code>print_system.sql</code>	Print settings keys + pharmacy-assets storage bucket
035	<code>phase16a_accounting_fixes.sql</code>	Widened <code>payment_type</code> CHECK, fixed <code>complete_grn</code> party linkage, proportional discount reversal in <code>process_return</code> , overpayment guard, dropped orphaned RPC overload
036	<code>bug_fixes.sql</code>	Seven fixes: report reversal double-count, expired-batch sale block, <code>audit_logs</code> immutability trigger, <code>medicines_select</code> RLS <code>is_deleted</code> filter, PO status transition trigger, <code>complete_sale</code> shift check, opening-balance unique index

Section 8 — Deployment

8.1 Environment Variables

Required (values intentionally omitted):

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
SUPABASE_JWT_SECRET
SUPABASE_DB_URL
DIRECT_URL
```

`SUPABASE_SERVICE_ROLE_KEY` must never be exposed to the client — it is used only in server-side scripts and server actions that explicitly need to bypass RLS.

8.2 Demo Environment

- **URL:** <https://pharmacare-demo-eight.vercel.app>
- **Supabase project:** `gnxtmvkfawfkmyyqebwi` (separate project from the development database — never share credentials between the two)
- Seeded with three months of realistic transaction data (April–June 2026) covering every major workflow: sales, returns, purchase orders, GRNs, supplier and customer payments, expenses, borrowing, a manual journal entry and its reversal, and an opening-balance entry.

8.3 Production Deployment Checklist

- Verify auth audit logging captures real IP address and user agent after the first real login on the production Vercel deployment — this could not be fully verified in local development, since there is no reverse proxy locally to inject the `x-forwarded-for` header. If `ip_address` / `user_agent` show as null in `audit_logs` for `LOGIN` / `LOGIN_FAILED` / `LOGOUT` rows, check the Vercel project's header configuration.
- Confirm all 7 environment variables (Section 8.1) are set in the target Vercel project's settings before deploying.

- Confirm the deployed database project matches the intended target — the demo and development databases are entirely separate Supabase projects, and running a migration or seed script against the wrong one would be a serious, hard-to-reverse mistake.

Section 9 — Known Bugs & Technical Debt

- `get_financial_summary()` **4900 contra-revenue bug** — see Section 5.3 in full. Recommended fix: special-case the 4900 balance (subtract rather than add) within the function's `account_type='revenue'` aggregation, mirroring how `get_balance_sheet()`'s NET calculation already does it correctly.
- **Legacy routes** — six route directories (`app/dashboard` , `app/expenses` , `app/inventory` , `app/reports` , `app/sales` , `app/bulk-upload`) predate the current Supabase-backed architecture and still read/write via `localStorage` , violating the project's own hard rule against `localStorage` for business data. They currently self-protect only by accident (their shared `AppLayout` reads a `localStorage` key that nothing in the current codebase writes, so any real user is redirected to `/login` on mount) — they are not in `proxy.ts`'s route-role map and must not be added there until fully rebuilt on Supabase.
- **Rate limiting deferred** — no rate-limiting package or middleware exists yet on `/login` or `/change-password` . Deferred deliberately for a single-branch, staff-only, no-self-signup deployment; required before any multi-branch or public-facing expansion (planned approach: Upstash Redis in middleware).
- **Sub-permission system deferred** — the current permission model is intentionally flat (18 permissions, no per-module view/edit/deactivate hierarchy). A hierarchical version is planned but explicitly deferred until a separate spec is written and approved.
- `next_po_number()` **is not transaction-safe** under concurrent inserts — acceptable at current single-branch PO volume, but a known limitation if concurrency increases.
- `purchase_order_items` **DELETE policy gap** — status guard (draft-only) is enforced at the application layer only, not RLS; a direct API call could bypass it (see Section 3.3).

Section 10 — Development Conventions

- **Soft delete everywhere.** Every table carries `is_deleted` / `deleted_at` (/ `deleted_by`); no business record is ever hard-deleted. `controlled_drug_register` and `audit_logs` go further — they have no delete path at all, not even soft-delete, because they are legally-oriented, append-only records.
- **Every write inserts an audit log entry.** This is treated as a hard rule, not a nice-to-have.

- **No `localStorage` for any business data.** All persistence goes through Supabase. (The six legacy routes in Section 9 predate and violate this rule, which is exactly why they are frozen and not extended.)
- **Server Actions for all mutations,** never client-side direct-to-database writes for anything that needs an audit trail or permission check.
- **Zod validation** at the server-action boundary for input parsing — never trust client-supplied data as pre-validated.
- **Money types** are `NUMERIC`, never `FLOAT` — balances in the accounting tables specifically use `NUMERIC(15,4)` for extra precision headroom; catalog-level prices use `NUMERIC(10,2)`.
- **Both RLS (database layer) and route/server-action guards (application layer)** are required together for every access-controlled feature — neither one alone is considered sufficient.
- **Test conventions:** Jest only; `--runInBand` whenever more than one journal-writing test file runs together; test data always tracked by exact ID for cleanup, never by name pattern, to avoid any risk of touching real records.